

Unit 5: Function, Exception and File Handling

5.1 Introduction to Functions

In Python, a **function** is a reusable block of code designed to perform a specific task. Functions help you organize your code, reduce redundancy, and make your programs more readable and maintainable.

Why Use Functions?

- To **avoid repetition** of code
- To **break down** complex problems into smaller parts
- To make code **modular** and **organized**
- To **reuse** code efficiently

Syntax:

```
def function_name():  
    # code block
```

Example:

```
def greet():  
    print("Hello! Welcome to Python.")
```

`greet()` # Output: Hello! Welcome to Python.

5.2 Defining and Calling Functions

In Python, **defining a function** allows you to create a reusable block of code that can be executed whenever you call (invoke) it.

Defining a Function

Use the `def` keyword followed by the function name and parentheses `()`:

```
def function_name():  
    # block of code
```

Example:

```
def say_hello():  
    print("Hello, World!")
```

Calling a Function

To execute the function, use its name followed by parentheses:

```
say_hello() # Output: Hello, World!
```

Function with Parameters

Functions can accept values (parameters) for dynamic operations:

```
def greet(name):  
    print("Hello,", name)
```

```
greet("Alice") # Output: Hello, Alice
```

5.3 Passing Arguments to Functions

In Python, **arguments** are values passed to functions when they are called. They allow you to send data to functions for processing.

Types of Arguments:

1. Positional Arguments

Values are assigned to parameters based on their position.

```
def greet(name, age):  
    print(f"Hello {name}, you are {age} years old.")
```

```
greet("Alice", 25)
# Output: Hello Alice, you are 25 years old.
```

2. Keyword Arguments

You specify the parameter names explicitly during the function call.

```
greet(age=30, name="Bob")
# Output: Hello Bob, you are 30 years old.
```

3. Default Arguments

Assign default values to parameters in the function definition.

```
def greet(name="User"):
    print(f'Hello {name}')
```

```
greet()      # Output: Hello User
greet("Alice") # Output: Hello Alice
```

4. Variable-Length Arguments

To handle an unknown number of arguments:

- ***args** (Non-keyword arguments):

```
def add_numbers(*numbers):
    total = sum(numbers)
    print("Sum:", total)
```

```
add_numbers(1, 2, 3) # Output: Sum: 6
```

- ****kwargs** (Keyword arguments):

```
def print_details(**info):
    for key, value in info.items():
        print(f'{key}: {value}')
```

```
print_details(name="Alice", age=25)
# Output: name: Alice
#       age: 25
```

5.4 Value-Returning Functions

In Python, **value-returning functions** are functions that send a result back to the part of the program that called them. This is done using the return statement.

Why Use return?

- To get the output of a function
- To pass data from one part of a program to another
- To perform further operations using the result

Syntax:

```
def function_name(parameters):
    # processing
    return value
```

Example:

```
def add(a, b):
    return a + b
```

```
result = add(5, 3)
print("Sum:", result)
```

Returning Multiple Values:

Python functions can also return multiple values using tuples:

```
def calculate(x, y):  
    return x + y, x - y
```

```
sum_val, diff_val = calculate(10, 4)  
print("Sum:", sum_val)  
print("Difference:", diff_val)
```

5.5 Introduction to File Input and Output (I/O)

File I/O in Python allows you to read from and write to files. This is useful for storing data permanently (unlike variables that are lost when the program ends).

Basic File Operations

Python provides built-in functions to handle file operations:

Mode	Description
'r'	Read (default)
'w'	Write (overwrites file)
'a'	Append
'x'	Create (error if exists)
'b'	Binary mode
't'	Text mode (default)

1. Writing to a File

```
with open("example.txt", "w") as file:  
    file.write("Hello, Python!\n")  
    file.write("Writing to a file.\n")
```

2. Reading from a File

```
with open("example.txt", "r") as file:  
    content = file.read()  
    print(content)
```

3. Appending to a File

```
with open("example.txt", "a") as file:  
    file.write("This is new content.\n")
```

4. Reading Line-by-Line

```
with open("example.txt", "r") as file:  
    for line in file:  
        print(line.strip())
```

5. Reading into a List

```
with open("example.txt", "r") as file:  
    lines = file.readlines()  
    print(lines)
```

6. Handling Files with Exception

```
try:  
    with open("example.txt", "r") as file:  
        print(file.read())  
except FileNotFoundError:  
    print("File not found.")
```

5.6 Using Loops to Process Files

Loops are commonly used to read and process files line by line, which is efficient for large files and allows for structured data handling.

Why Use Loops?

- To read each line individually
- To process or filter content

- To save memory (especially with large files)

Using a for Loop to Read File Line-by-Line

```
with open("example.txt", "r") as file:  
    for line in file:  
        print(line.strip()) # .strip() removes newline characters
```

Use of File in Python Programs

Files are used in Python to **store, read, and write data permanently**. Unlike variables, which hold data temporarily in memory, files allow **persistent storage** across program executions.

5.7 Exception Handling in Python

What is an Exception?

An **exception** is an error that occurs during program execution, which disrupts the normal flow of the program. If not handled properly, the program will crash.

Examples of common exceptions:

- `ZeroDivisionError`: Division by zero
- `FileNotFoundError`: Trying to open a file that doesn't exist
- `ValueError`: Invalid value (e.g., converting a string to int)
- `IndexError`: Accessing an out-of-range list index

How to Handle Exceptions in Python

Exception handling in Python is done using the try-except block. It allows you to catch and handle errors during program execution without crashing the program.

Syntax:

```
try:  
    # Code that may raise an exception  
    x = int(input("Enter a number: "))  
    result = 10 / x  
    print(result)
```

```
except ZeroDivisionError:  
    print("Error: You cannot divide by zero.")  
except ValueError:  
    print("Error: Invalid input. Please enter a number.")
```